

ICS 26 Fall Lab1 Data Lab 实验报告

check_ops.py 检查结果

```
● longxq@User-2025YVAIWL:~/ICS_Lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 22/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 48/48 operators
mul5Sat: 30/30 operators
classifyAdd3: 41/52 operators
floatScaleThreeHalves: 51/60 operators
floatRoundEven: 18/65 operators
float_i2f: 26/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

btest 功能测试结果

```
longxq@User-2025YVAIWL:~/ICS_Lab1$ ./btest
Score  Rating  Errors  Function
1       1       0      signMask
2       2       0      bitXor
3       3       0      negativePart
4       4       0      copyByteWithin
4       4       0      logicalShift
4       4       0      swapNibblePairs
4       4       0      secondLowestZeroBit
5       5       0      oddParity
5       5       0      rotateRightBits
5       5       0      roundEvenPow2
5       5       0      midpointTowardFirst
7       7       0      isBetweenEitherOrder
7       7       0      mul5Sat
7       7       0      classifyAdd3
7       7       0      floatScaleThreeHalves
10      10      0      floatRoundEven
10      10      0      float_i2f
10      10      0      bitCount
10      10      0      bitReverse
Total points: 110/110
```

各题实现思路

P1 signMask:

题目：返回只有最高位为 1 的掩码 0x80000000。

思路：直接将整数 1 左移 31 位：1 << 31。二进制下 1 在最低位，左移 31 位后恰好到达最高位（第 31 位），其余位全为 0。

P2 bitXor:

题目：只用 ~ 和 & 实现异或 x^y 。

思路：首先得知异或的定义：两位不同则为 1，即 (x 为 1 且 y 为 0) 或 (x 为 0 且 y 为 1)；然后用德摩根定律展开：

$$x^y = (x|y) \& \sim(x\&y)$$

再把 | 展开： $x|y = \sim(\sim x \& \sim y)$

最终可以得到： $\sim(\sim x \& \sim y) \& \sim(x \& y)$ 。

P3 negativePart:

题目: $x < 0$ 返回 $-x$, 否则返回 0。

思路: 首先构造符号掩码 mask: 算术右移 $x \gg 31$: x 为负时补 1, 得到全 1 (0xFFFFFFFF); x 非负时补 0, 得到全 0。

然后返回 $\text{mask} \& (\sim x + 1)$, 最后得到:

- x 负时: mask 全 1, $\sim x + 1 = -x$, 结果就是 $-x$

- x 非负时: mask 全 0, 任何数 $\& 0 = 0$

P4 copyByteWithin:

题目: 把 x 的第 src 字节复制到第 dst 字节。

思路: 字节编号转位偏移: src 字节 = $\text{src} \times 8$ 位, 所以偏移量 $s = \text{src} \ll 3$, $d = \text{dst} \ll 3$ 。

然后取出源字节: $(x \gg s) \& 0xFF$, 即右移 s 位把源字节移到最低位, 再 $\& 0xFF$ 取出来。

接着构造目标掩码: $0xFF \ll d$, 也就是在目标位置上的 8 位掩码。

接下来清除目标位: $x \& \sim \text{mask}$ 。

再放入新字节: $(\text{byte} \ll d)$, 并把源字节移到目标位置。

最后用 $|$ 合并: $(x \& \sim \text{mask}) | (\text{byte} \ll d)$ 。

P5 logicalShift:

题目: 实现逻辑右移 (高位补 0)。

思路: C 语言的 $\gg$ 是算术右移, 负数会补 1。需要把补的 1 清掉。

先使用算术右移 $x \gg n$, 此时高 n 位是补的符号位。

构造掩码: $\sim((1 \ll 31) \gg n \ll 1)$

- $(1 \ll 31) = 0x80000000$

- $\gg n$ 后高 n 位补 1, 变成 $0xFFFF...$ (高 n 位为 1)

- $\ll 1$ 左移回来, 高 n 位还是 1, 但最低位变 0

- $\sim$ 取反后, 高 n 位变 0, 其余位全 1

然后 $(x \gg n) \&$ 这个掩码, 就把高 n 位的 1 清掉了。

思考: 一开始想直接 $\sim(0xFFFFFFFF \gg n)$, 但 $0xFFFFFFFF$ 是大常数不合法。后来想到用 $(1 \ll 31) \gg n$ 来生成高位 1 的掩码, 再取反。这个技巧后来在 P9 循环右移里也复用了。

P6 swapNibblePairs:

题目: 交换每个字节内的高低 4 位。

思路: 构造掩码 $0x0F0F0F0F$: $m = 0x0F$, $m |= m \ll 8$, $m |= m \ll 16$ 。

这个掩码的特点是: 每字节的低 4 位为 1, 高 4 位为 0。

然后: $((x \& m) \ll 4) | ((x \gg 4) \& m)$

- $x \& m$ 取出所有低 4 位, $\ll 4$ 移到高 4 位位置

- $x \gg 4$ 把高 4 位移到低 4 位位置，再 $\&m$ 取出来
- $|$ 合并两部分

P7 secondLowestZeroBit:

题目：返回标记第二个最低位 0 位置的掩码。

思路：找最低位 0 的技巧： $\sim x \& (x + 1)$

- $x+1$ ：从最低位开始，遇到第一个 0 就把它变成 1，后面的 1 全变 0

- $\sim x$ ：把 0 变 1，1 变 0

- $\&$ 后，恰好只有那个最低位 0 的位置为 1

例： $x = 0xFFFFFFFFA$ (...11111010)， $x+1 = 0xFFFFFFFFB$ ， $\sim x = 0x00000005$ ， $\& = 0x4$ ，就是最低位 0 的位置。

第二步：把这个 0 变成 1： $x2 = x | z1$

第三步：在 $x2$ 上再找最低位 0，就是原来的第二个最低位 0。

思考：一开始没想到 $\sim x \& (x+1)$ 这个经典技巧，卡了很久。知道了找最低位 0 的方法后，第二步把它变成 1，再找一次就好了。

P8 oddParity:

题目：返回奇偶校验位：1 的个数为偶数返回 1。

思路：异或的性质： $a^a=0$ ， $a^0=a$ 。所有位的异或结果就是 1 的个数的奇偶性。

分治法并行计算：

$x^{\wedge} = x \gg 16$ —— 高 16 位和低 16 位异或，结果存在低 16 位

$x^{\wedge} = x \gg 8$ —— 高 8 位和低 8 位异或

$x^{\wedge} = x \gg 4$ —— 再压缩

$x^{\wedge} = x \gg 2$

$x^{\wedge} = x \gg 1$

最后低 1 位就是所有 32 位的异或结果。

如果异或结果为 0（1 的个数为偶数）， $!0=1$ ；否则为 0。

P9 rotateRightBits:

题目：循环右移 n 位。

思路：循环右移 = 低 n 位移到高位 + 高位右移到低位。

先取 $n = n \& 31$ ，限制在 0~31。

高位移出部分： $high = (x \gg m)$ ，但要清掉算术右移补的符号位（用 P5 的掩码技巧）。

低位移到高位： $low = x \ll (32 - n)$ 。32-n 用 $(32 + \sim n + 1)$ 实现。

最后 $high | low$ 合并。

思考：复用了 P5 的掩码技巧。另外减法不能直接用，要用 $\sim+1$ 来实现。

P10 roundEvenPow2:

题目：四舍五入到最近的 2^n 倍数，halfway 选偶数商。

思路：设 $m = 1 \ll n$, $h = m \gg 1$ (半值)。

取低 n 位: $low = x \& (m-1)$ 。

判断:

- $low > h$: 向上取整 (加 m)

- $low < h$: 向下取整 (不加)

- $low == h$: halfway, 看第 n 位 (当前商的奇偶)。如果商为奇数 ($bitN=1$)，向上取整 (选偶数商)；如果商为偶数，不动。

$delta = (gt \& m) | ((tie \& bitN) \ll n)$

最后 $(x + delta) \& \sim(m-1)$ 清掉低 n 位。

思考：halfway 选偶数这个条件一开始没搞懂，想清楚是选商为偶数的那个之后就好办了。另外不能直接用减法比较 low 和 h ，要用 $(h - low) \gg 31$ 判断大小。

P11 midpointTowardFirst:

题目：计算 $(x+y)/2$ ，不溢出，halfway 选靠近 x 的。

思路：直接 $x+y$ 会溢出。用位运算安全计算：

$floor = (x \& y) + ((x \oplus y) \gg 1)$

原理: $x \& y$ 是两数都为 1 的位 (这些位相加会进位)， $x \oplus y$ 是只有一个为 1 的位 (不进位)。

$(x \oplus y) \gg 1$ 就是不进位的那部分的平均， $x \& y$ 是进位部分。

然后判断 halfway: x 和 y 的和为奇数时才需要选择。

$odd = (x \oplus y) \& 1$ 。

如果和为奇数，且 $x \geq y$ ，选更大的那个 (靠近 x)：

gt = 判断 $x \geq y$ (同号用减法，异号用符号)。

结果 = $floor + (odd \& gt)$ 。

思考： $(x \& y) + ((x \oplus y) \gg 1)$ 这个技巧很巧妙，一开始没想到。另外判断 $x \geq y$ 的时候，异号情况直接用符号判断，同号才用减法。

P12 isBetweenEitherOrder:

题目：判断 x 是否在 $[a,b]$ 区间内 (a,b 顺序任意)。

思路：分别判断 $x \geq a$ 、 $x \geq b$ 、 $x \leq a$ 、 $x \leq b$ 。

判断 $x \geq y$ 的方法:

- 如果 x 和 y 异号: x 正 y 负则 $x \geq y$, x 负 y 正则 $x < y$

- 如果同号: $(x-y) \gg 31$ 为 0 则 $x \geq y$

用符号位 sx,sa,sb , 异号判断 $opp = sx \oplus sa$ 。

$geXA = (oppXA \& \sim sx) \mid (\sim oppXA \& \sim ((x-a) >> 31))$

最后组合: $(x >= a \&\& x <= b) \mid (x >= b \&\& x <= a)$ 。

思考: ops 卡了很久, 最后把重复的 $\sim sx$ 提出来省了一个 op, 从 50 压到 48。

P13 mul5Sat:

题目: 计算 $x*5$, 溢出时饱和到 INT_MAX/INT_MIN。

思路: 先算 $x*4 = x << 2$, 再加 x 得 $t = x*5$ 。

判断溢出有两种情况:

1. 加法溢出: $x*4$ 和 x 同号, 但结果 t 变号了。

$addOv = \sim((x << 2) \wedge sx) \& ((t >> 31) \wedge sx)$

2. 乘法溢出但加法没溢出: x 太大, 乘以 4 就已经溢出了。

正数: $x >> 29 \neq 0$ (x 太大, 左移 2 位就溢出)

负数: $\sim x >> 29 \neq 0$

$big = (\sim sx \& !(x >> 29)) \mid (sx \& !(\sim x >> 29))$

总溢出 $ov = addOv \mid big$ 。

溢出时: 正数取 INT_MAX, 负数取 INT_MIN。

$sat = sx \wedge \sim(1 << 31)$ —— sx 为负时 $sat = INT_MIN$, sx 为正时 $sat = INT_MAX$ 。

思考: 一开始只考虑了加法溢出, 漏掉了 $x*4$ 本身就溢出的情况 (比如 $x = 0x40000000$, $x*4$ 就溢出了, 但加 x 的时候反而没溢出)。另外 big 变量里 $x >> 29$ 可能返回 2、3 等值, 加了!!归一化之后才对。

P14 classifyAdd3:

题目: 判断 $x+y+z$ 的和: $>INT_MAX$ 返回 1, $<INT_MIN$ 返回 -1, 否则 0。

思路: 分步加法: $s = x+y$, $t = s+z$ 。

判断每步的溢出:

$ov1_pos$: x, y 都正但 s 变负

$ov1_neg$: x, y 都负但 s 变正

$ov2_pos$: s, z 都正但 t 变负

$ov2_neg$: s, z 都负但 t 变正

组合时注意: 第一步正溢出后第二步负溢出不算真溢出 (先正溢再负溢, 结果可能拉回来了)。

$pos = (ov1_pos \& \sim ov2_neg) \mid (\sim ov1_pos \& \sim ov1_neg \& ov2_pos)$

neg 同理。

最后用符号掩码把 pos/neg 转成 1/-1/0。

思考: 关键是想清楚分步溢出的组合关系: 第一步溢出后第二步可能把结果拉回来, 不能简单地 OR 起来。画了个真值表才想明白。

P15 floatScaleThreeHalves:

题目：计算浮点数 $f * 3/2$ 。

思路：拆出 s （符号）、 e （阶码）、 f （尾数）。

$e == 255$ 是特殊值（NaN/无穷），直接返回。

非规格化（ $e == 0$ ）： $M = f$, $E = -126$ 。

规格化： $M = [1 < 23]f$, $E = e - 127$ 。

然后 $M * 3 = M + M + M$ 。

重新规范化：

- 如果 $M3 \geq 4$ （即 $\geq 1 < 25$ ），右移 2 位，阶码+2

- 如果 $M3 \geq 2$ （即 $\geq 1 < 24$ ），右移 1 位，阶码+1

- 如果 $M3 < 1$ （即 $< 1 < 23$ ），左移直到规范化，阶码递减

然后 round-to-nearest-even: $\text{tail} > \text{half}$ 或 $\text{tail} == \text{half}$ 且 $\text{lsb} = 1$ 时进位。

下溢（ $E < -126$ ）时右移成非规格化数。

上溢（ $E \geq 128$ ）时返回无穷。

思考：一开始忘了处理非规格化下溢的情况，直接返回 0 导致测试用例过不了。后来想清楚下溢时要把尾数右移成非规格化数，而不是直接返回 0。

P16 floatRoundEven:

题目：浮点数四舍五入到整数，halfway 选偶数。

思路：拆出 s, e, f 。 $e == 255$ 直接返回。

$\text{exp} = e - 127$ 。

- $\text{exp} \geq 23$ ：小数部分为 0，直接返回原数

- $\text{exp} < -1$ ： $|x| < 0.5$ ，返回 ± 0

- $\text{exp} == -1$ ：正好 0.5 附近， > 0.5 返回 ± 1.0 ， ≤ 0.5 返回 ± 0

- $0 \leq \text{exp} < 23$ ：需要舍掉低 $\text{nfrac} = 23 - \text{exp}$ 位小数

取出 frac 和 half 比较。

如果 $\text{frac} > \text{half}$ 或 $\text{frac} == \text{half}$ 且 $\text{lsb} = 1$ （含隐含的 1），进位。

思考： lsb 一开始忘了加隐含的 1（即 $0x800000$ ），导致 halfway 判断不对。后来想清楚规格化数的尾数前面有个隐含的 1， lsb 要把它算进去。

P17 float_i2f:

题目：int 转 float 的位级表示。

思路： $x == 0$ 直接返回 0。

取符号 s ，取绝对值 $n = -x$ （ INT_MIN 也安全）。

找最高位 k ：从 31 往下找第一个为 1 的位。

阶码 $e = k + 127$ 。

- $k \leq 23$: 尾数不够 23 位, 左移补 0: $f = n \ll (23 - k)$
- $k > 23$: 尾数超过 23 位, 右移并 round-to-nearest-even
 $f = n \gg (k - 23)$, tail 和 half 比较决定是否进位。
进位后如果 $f \geq 1 \ll 24$, 阶码+1, f 清 0。

思考: INT_MIN 取绝对值的时候要注意, 不能直接取负, 要用无符号减法。另外进位后尾数可能变成 24 位, 这时候要阶码+1, 尾数清 0。

P18 bitCount:

题目: 计算 x 中 1 的个数。

思路: 分治法并行计数:

第一步: 每两位相加。 $(x \& 0x55555555) + ((x \gg 1) \& 0x55555555)$, 结果每两位存这两位 1 的个数。

第二步: 每四位相加。 $(x \& 0x33333333) + ((x \gg 2) \& 0x33333333)$, 每四位存这四位的 1 的个数。

第三步: 每八位相加。 $(x \& 0x0F0F0F0F) + ((x \gg 4) \& 0x0F0F0F0F)$ 。

然后 $x += x \gg 8$ (高 16 位加到低 16 位), $x += x \gg 16$ (高 16 位加到低 16 位)。

最后 $\& 0x3F$ 取出低 6 位 (最多 32 个 1, 6 位足够)。

P19 bitReverse:

题目: 反转 32 位整数的所有位。

思路: 从大到小分治交换:

第一步交换高低 16 位:

$m = 0x0000FFFF$ ($m = 0xFF$; $m |= m \ll 8$)

$x = (x \ll 16) | ((x \gg 16) \& m)$

第二步交换相邻 8 位:

$m = m \wedge (m \ll 8)$ —— 从 $0x0000FFFF$ 递推到 $0x00FF00FF$

$x = ((x \& m) \ll 8) | ((x \gg 8) \& m)$

第三步交换相邻 4 位:

$m = m \wedge (m \ll 4)$ —— 递推到 $0x0F0F0F0F$

$x = ((x \& m) \ll 4) | ((x \gg 4) \& m)$

第四步交换相邻 2 位:

$m = m \wedge (m \ll 2)$ —— 递推到 $0x33333333$

$x = ((x \& m) \ll 2) | ((x \gg 2) \& m)$

第五步交换相邻 1 位:

$m = m \wedge (m \ll 1)$ —— 递推到 $0x55555555$

$x = ((x \& m) \ll 1) | ((x \gg 1) \& m)$

关键技巧: 用 $m = m \wedge (m \ll n)$ 从已有掩码递推下一级掩码, 而不是每次重新构造, 节省了大量运算符。

思考：一开始是从小到大做的（先交换相邻位，再交换 2 位...），每次都要重新构造掩码，ops 数超了 6 个。后来改成从大到小做，并且用 $m = m \oplus (m < n)$ 递推掩码，正好压到 34 ops。这个递推掩码的技巧是关键突破。